Calculabilitate

- Dat fiind un graf G și două noduri S și D ..
  - Problemă de decizie
    - **Există** un drum de la S la D? (YES/NO)
  - Problemă de căutare
    - **Descoperă** un drum de la S la D.
  - Problemă de optimizare
    - Întoarce **cel mai scurt** drum de la S la D.
  - Problemă de generare
    - Întoarce **toate** drumurile de la S la D.

Notăm:

- Fie $f_Q : \mathbb{N} \to \{1, 0\}$, specificația problemei de decizie Q.
- Fie $A = \{x \in \mathbb{N} \mid f_Q(x) = 1\}$, mulțimea de adevăr asociată.

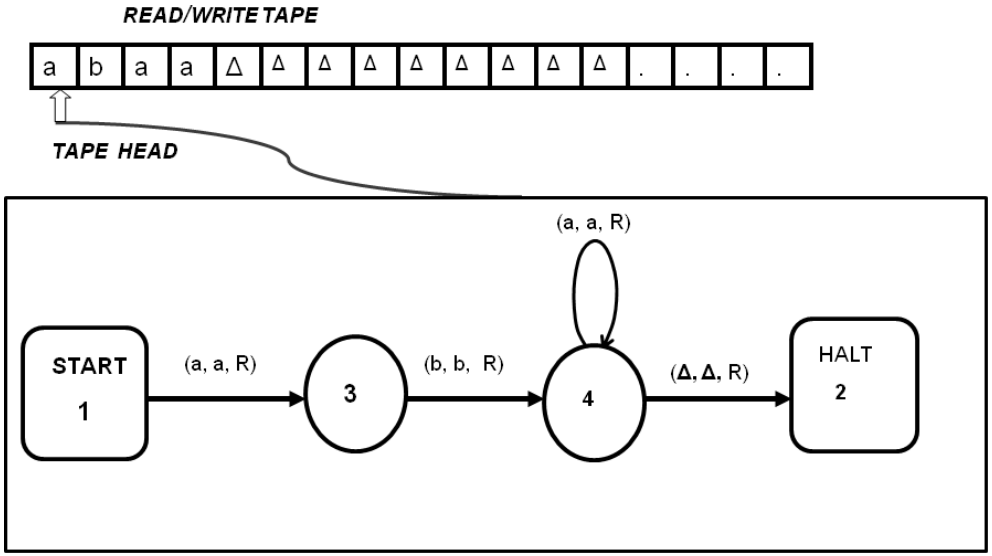


Problemă Funcție **Algoritm** Program

- Ce este un algoritm/o *procedură efectivă*?

- Număr finit de instrucțiuni, specificate fără ambiguitate.
- Procesul de calcul necesită un număr **finit** de pași.
- Procesul de calcul necesită o cantitate **finită** de memorie.
- Produce un rezultat neambiguu, mereu același, pentru orice intrare validă.

- Modele de calcul - **Mașina Turing**



Exemplu Mașină Turing[1]

Pe fiecare muchie avem (*simbol curent*, *simbol înlocuitor*, *direcția de mutat capul*)

[1]Animație MT: http://www.asethome.org/mathfoundations/tmd/

- Modele de calcul - **Mașina Turing** (not. MT)

- Q - mulțimea de stări
  - $q_{init} \in Q$ - starea inițială
  - $q_{accept} \in Q$ - starea care semnifică terminarea cu succes (*acceptarea* intrării)
  - $q_{reject} \in Q$ - starea care semnifică eșuarea procesului de calcul (*respingerea* intrării)
- Σ - alfabetul (valorile care pot fi scrise pe bandă, inclusiv simbolul nul)
- $\delta : Q \times \Sigma \rightarrow Q \times \sigma \times \{L, R\}$



- Alte exemple de modele de calcul
  - Calculul Lambda
  - Funcțiile recursive ale lui Kleene

- Deoarece putem reprezenta o MT ca un string, mulțimea tuturor MT este numărabilă.
- Putem defini o MT *universală* care să simuleze orice altă MT, a cărei codificare este primită ca argument la intrare.
- Recursion Theorem: Fie T, o MT care calculează o funcție $f : \Sigma^* x \Sigma^* \rightarrow \Sigma^*$. Atunci există o MT R, care calculează $r : \Sigma^* \rightarrow \Sigma^*$, astfel încât, pentru $\forall w$[2]:

$$T(<R>, w) = R(w)$$

.

[2]Notăm cu $<R>$ descrierea mașinii Turing R.

Tipuri de funcții

Funcție efectiv-calculabilă

O funcție este efectiv-calculabilă dacă există un algoritm / o procedură efectivă de rezolvare a problemei.

Funcție recursivă

O funcție este calculabilă / *recursivă* în sens Turing dacă există o mașină Turing care se termină în timp finit și returnează răspunsul corect pentru orice intrare validă, respectiv ciclează pentru intrări invalide.

Teza Church-Turing

O funcție este efectiv calculabilă $\iff$ funcția este calculabilă în sens Turing.

Tipuri de mulțimi

A - Mulțime recursivă (R)

Există un program care poate verifica, în timp **finit**, dacă un element aparține **sau nu** mulțimii.

$$A \in R \iff P(x) = \begin{cases} 1, & x \in A \\ 0, & x \notin A \end{cases}$$

Tipuri de mulțimi

Exemple de mulțimi recursive (R)

- $\{x \in \mathbb{N} \mid$ x este un număr prim$\}$
- $\{x \in \mathbb{N} \mid P_x(w)$ se termină în cel mult t unități de timp$\}$[1]

[1] Folosim indicele x pentru a identifica un program specific din mulțimea infinit numărabilă a tuturor programelor.

Tipuri de mulțimi

A - Mulțime recursiv-enumerabilă (RE)

Există un program care poate confirma în timp finit dacă un element aparține mulțimii A, dar poate cicla la infinit dacă elementul **NU** aparține mulțimii.

$$A \in R \iff P(x) = \begin{cases} 1, & x \in A \\ \perp, & x \notin A \end{cases}$$

Tipuri de mulțimi

A - Mulțime recursiv-enumerabilă (RE)

Alternativ: Elementele din mulțimea A pot fi enumerate. Cu alte cuvinte, există un program G[a] care la fiecare apel generează succesiv câte un element nou din A.

- Orice element din A va fi returnat după un număr **finit** de apeluri.
- Dacă A este finită, G poate să cicleze după generarea ultimului element.

[a] G este un *generator* al mulțimii A.

Tipuri de mulțimi

Observații:

- Cele două definiții pentru o mulțime RE sunt echivalente.
- O mulțime recursivă este și recursiv-enumerabilă.
- Reciproca nu este în general valabilă.

Tipuri de mulțimi

Exemple de mulțimi recursiv-enumerabile (RE)

- $\{x \in \mathbb{N} \mid$ Programul $P_x(w)$ se termină$\}$
- $\{x \in \mathbb{N} \mid \exists y_1, .., y_k \in \mathbb{N} \rightarrow p(x, y_1, .., y_k) = 0\}$, unde p este un polinom cu coeficienți numere întregi [Dav73]

Tipuri de mulțimi

Exemple de mulțimi care NU sunt RE

- $\{x \in \mathbb{N} \mid$ Programul $P_x(w)$ **NU** se termină$\}$
- $\{x, y \in \mathbb{N} \mid \forall w. P_x(w) = P_y(w)\}$

- Fie A mulțimea de adevăr a problemei de decizie Q. Atunci, în funcție de tipul lui A, Q este:

A	Q
R	Decidabilă
RE	Semi-decidabilă [Jon02][2]
!R	Nedecidabilă

[22] Folosim această terminologie din rațiuni didactice, dar, în alte lucrări din domeniu, puteți întâlni termenul de semidecidabilitate folosit pentru a descrie toate problemele a căror mulțime de adevăr este RE dar nu este R. [Giu]

Reducere (many-one)

- Problem A se reduce la o problema B ($A \leqslant_m B$)

dacă și numai dacă

- $\exists$ o funcție calculabilă F care transformă $\forall$ intrare i a problemei A într-o intrare $F(i)$ pentru problema B, astfel încât $A(i) = 1 \iff B(F(i)) = 1$.

Reducere (many-one) - Observații

- Informal, $A \leqslant_m B$ ne sugerează că B reprezintă o instanță mai generală a problemei A.
  - B este "cel puțin la fel de grea" ca A.
- Relația nu este simetrică:
  - Dacă A se reduce la B, nu înseamnă că și B se reduce la A.
- Pentru a demonstra $A \leqslant_m B$ trebuie să arătăm simultan:
  - $A(i) = 1 \implies B(F(i)) = 1$
  - $B(F(i)) = 1 \implies A(i) = 1$
- Relația este reflexivă, antisimetrică și tranzitivă.

Reducere (many-one) - Consecințe

- Fie două probleme de decizie Q_A și Q_B, astfel încât $Q_A \leqslant_T Q_B$. Fie A și B mulțimile de adevăr asociate celor două probleme. Atunci:
  - Dacă B este R $\Rightarrow$ A este R.
  - Dacă B este RE $\Rightarrow$ A este RE.
  - Dacă A nu este R $\Rightarrow$ B nu este R.
  - Dacă A nu este RE $\Rightarrow$ B nu este RE.

Reducere Turing

Oracol

Definim un oracol pentru o problemă un program care poate să rezolve în timp finit orice instanță a problemei.

Reducere Turing

Spunem că problema A se reduce Turing la B (not. $A \leqslant_T B$) dacă, având un oracol pentru problema B (**pe care îl putem apela de oricâte ori avem nevoie**), putem calcula în timp finit răspunsul pentru orice instanță a problemei A.

Exemple de probleme nedecidabile (1)

- Halting Set: $\{x \in \mathbb{N} \mid \text{Programul } P_x(w) \text{ se termină}\}$

Halting Set - Demonstrație (1)

- Presupunem prin absurd că există un program *Halt* care verifică în timp finit dacă programul cu indicele i se termină pe intrarea w:

$$Halt(i, w) = \begin{cases} True & \text{dacă apelul } P_i(w) \text{ se termină} \\ False & \text{otherwise} \end{cases}$$

Halting Set - Demonstrație (2)

- Atunci, putem construi următorul program:

```
1: function ReverseHalt(i)
2:     if Halt(i, i) then
3:         while (1);
4:     else
5:         return True;
6:     end if
7: end function
```

- Fie j, indicele programului *ReverseHalt* din enumerarea tuturor programelor.
- Ce se întâmplă când apelăm *ReverseHalt*(j)?

Busy-Beaver

- Fie funcția $BB : \mathbb{N} \to \mathbb{N}$[3]:

$$BB(n) = max(\{P() \mid |P| \leqslant n \text{ și } P() \text{ se termină}\})$$

- $BB(n)$ întoarce valoarea maximă a răspunsului produs de un program P, **a cărui lungime este mai mică decât** n.

[3] Definim $max(\varnothing) = 0$.

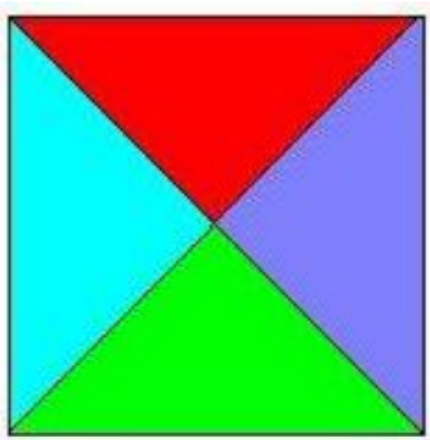
Teorema lui Rice

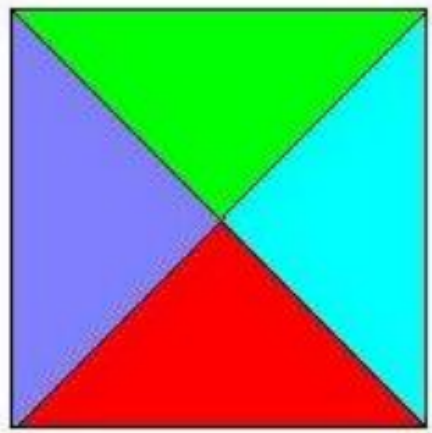
- Fie Q o proprietate a programelor:
  - **Semantică**/Extensională:
    - Dacă două programe P_A și P_B sunt echivalente semantic[4][5], atunci fie ambele au proprietatea Q, fie nu o au.
  - **Netrivială**
    - Q aparține cel puțin unui program dar nu tuturor programelor.
- Atunci problema determinării dacă un program are proprietatea Q este **nedecidabilă**.

[4] Dacă primesc aceeași valoare la intrare, produc același răspuns.

[5] Prop. Q se referă strict la relația dintre intrarea programului și ieșirea acestuia.

Tile Cover





- Date fiind o mulțime T de tupluri $< n, s, w, e >$, există o funcție recursivă totală $f : \mathbb{N} \times \mathbb{N} \to T$ astfel încât[6]:

$$f(i,j)[n] = f(i+1,j)[s]$$
$$f(i,j)[e] = f(i,j+1)[w]$$
$$f(0,0) = c(\text{corner tile})$$

[6]Source: `http://www.cs.bc.edu/~straubin/cs385-07/tiling`

Mortal Matrix

- Dat fiind N matrici de dimensiuni $D \times D$, cu elemente întregi, există o secvență finită de operații de înmulțire (posibil repetată) a acestora care produc matricea zero?

Post Correspondence Problem

- Dându-se 2 liste W și X a câte N cuvinte fiecare, există o secvență nevidă de k întregi $i_1, i_2, ..., i_k$ astfel încât $w_{i_1} w_{i_2} ... w_{i_k} = x_{i_1} x_{i_2} ... x_{i_k}$?

- Exemplu: $W = [a, bb, a]$, $X = [aa, b, bb]$.
- Soluție: $w_1 w_3 w_2 w_2 = x_1 x_3 x_2 x_2$
  - Ambele secvențe produc 'abba'.

Jocuri de echipă cu informație incompletă



- Există o strategie care poate garanta victoria unui jucător? [HD09, CL20]

Sursa imaginii

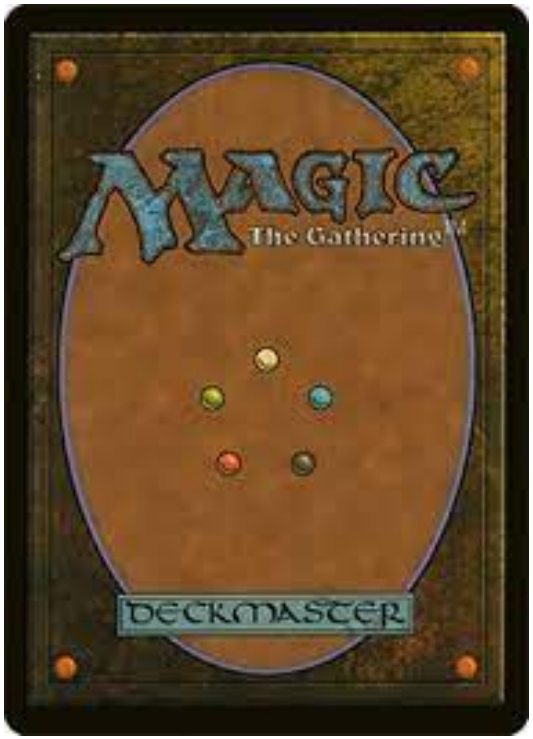
Jocuri de echipă cu informație incompletă



- Există o strategie care poate garanta victoria unui jucător? [HD09, CL20]

Sursa imaginii

Magic the Gathering



- Există o strategie care poate garanta victoria unui jucător? [CBH19]

[6] Sursa imaginii

Decidabilitate - Consecințe

Nu putem verifica automat, pentru **orice** program:

- Dacă rezolvă o anumită problemă.
- Dacă este echivalent cu un alt program.
  - Dacă păstrează funcționalitatea oferită de o versiune a sa mai veche.
- Dacă va avea un comportament malițios.
- Dacă o secvență de cod din program va fi executată.
- Dacă o secvență de cod din program poate fi executată în paralel fără a afecta rezultatul final.

Strategii de abordare a unei probleme nedecidabile

- Principala dificultate asociată provine din faptul că nu putem explora un spațiu **infinit** de posibilități fără a avea o informație suplimentară care să ne permită să restrângem spațiul de căutare.

- Prin urmare, vom accepta o rezolvare parțială care poate să rateze unele soluții corecte dar acoperă cazurile importante, luând eventual în considerare probabilitatea ca acestea să fie întalnite în practică.

Bibliografie I

- Alex Churchill, Stella Biderman, and Austin Herrick, *Magic: The gathering is turing complete*, arXiv preprint arXiv:1904.09828 (2019).
- Michael J. Coulombe and Jayson Lynch, *Cooperating in video games? impossible! undecidability of team multiplayer games*, Theoretical Computer Science **839** (2020), 30–40.
- Martin Davis, *Hilbert's tenth problem is unsolvable*, The American Mathematical Monthly **80** (1973), no. 3, 233–269.
- Cristian A. Giumale, *Introducere în analiza algoritmilor, cap. 4.5*.

Bibliografie II


Robert A Hearn and Erik D Demaine, *Games, puzzles, and computation*, CRC Press, 2009.


Neil D Jones, *Computability and complexity from a programming perspective*, Proof and System-Reliability (2002), 79–135.